HRMS-API

Human Resource Management System

Technical Documentation & API Reference

Framework:	ASP.NET Web API 2 / .NET Framework 4.8
Database:	SQL Server / Entity Framework 6
Auth:	JWT Bearer Tokens + Refresh Tokens
Version:	1.0.0

Table of Contents

1. Project Overview
2. System Architecture
    - 2.1 Architectural Pattern
    - 2.2 Solution Structure
    - 2.3 Technology Stack
3. Database Schema
    - 3.1 Entity Relationship Overview
    - 3.2 Table Definitions
4. Authentication & Security
    - 4.1 JWT Flow
    - 4.2 Roles & Permissions
    - 4.3 Token Strategy
5. API Endpoints
    - 5.1 Auth
    - 5.2 Users
    - 5.3 Departments
    - 5.4 Employees
    - 5.5 Salaries
    - 5.6 Bonuses
    - 5.7 Deductions
    - 5.8 Payroll
6. Business Logic
    - 6.1 Payroll Calculation
    - 6.2 Salary Revision
    - 6.3 Auto User Creation
7. Configuration & Setup
    - 7.1 Prerequisites
    - 7.2 Database Setup
    - 7.3 JWT Settings
    - 7.4 CORS
8. Example Usage
9. Technologies Used

1. Project Overview

HRMS-API is a role-based Human Resource Management System Web API built with ASP.NET Web API 2 on the .NET Framework 4.8 platform. It provides a clean, structured backend for managing all core HR operations including employee records, departmental structure, salary management, bonuses, deductions, payroll generation, and secure authenticated user access via JWT.

Key capabilities at a glance:

Authentication	JWT access tokens, refresh token rotation, logout, password change
User Management	Role assignment (Admin / HR / Employee), toggle active status
Employee Management	Full CRUD, keyword search, department and status filtering
Salary Management	Active salary lookup, revisions with full audit history
Bonus & Deductions	Per-employee tracking, period-based queries
Payroll	Monthly generation, approval workflow, mark-as-paid lifecycle

2. System Architecture

2.1 Architectural Pattern

The application follows a strict N-Tier layered architecture with three clearly separated projects. Each layer depends only on the layer directly below it, ensuring loose coupling and high cohesion.

Layer	Project	Responsibility
Presentation	API	HTTP Controllers, Routing, JWT Filter, CORS
Business Logic	BLL	Services, DTOs, AutoMapper Config, JWT Helper, BCrypt
Data Access	DAL	EF6 DbContext, Entities, Repositories, Migrations, DataAccessFactory
Database	SQL Server	Persistent storage — MSSQL via Entity Framework Code First

Request Flow

Step	Component	Action
1	Client (Angular / Postman)	Sends HTTP request with Bearer token
2	JWTAuthFilter (API)	Validates token, checks role, attaches ClaimsPrincipal
3	Controller (API)	Routes request, calls appropriate Service method
4	Service (BLL)	Applies business rules, validation, calls Repository
5	Repository (DAL)	Executes EF6 query against SQL Server
6	AutoMapper (BLL)	Maps Entity → DTO
7	Controller (API)	Returns HttpResponseMessage with DTO as JSON

2.2 Solution Structure

```
HRMS-API/
■■■ API/                          ← ASP.NET Web API 2 project
■   ■■■ Controllers/
■   ■   ■■■ AuthController.cs
■   ■   ■■■ UserController.cs
■   ■   ■■■ DepartmentController.cs
■   ■   ■■■ EmployeeController.cs
■   ■   ■■■ SalaryController.cs
■   ■   ■■■ BonusController.cs
■   ■   ■■■ DeductionController.cs
```

```
■   ■   ■■■ PayrollController.cs
■   ■■■ Filters/
■   ■   ■■■ JWTAuthFilter.cs
■   ■■■ App_Start/
■       ■■■ WebApiConfig.cs
■■■ BLL/                          ← Business Logic Layer
■   ■■■ DTOs/
■   ■■■ Services/
■   ■■■ Helpers/
■       ■■■ JWTHelper.cs
■       ■■■ MapperConfig.cs
■       ■■■ AppConfig.cs
■       ■■■ Roles.cs
■■■ DAL/                          ← Data Access Layer
    ■■■ Models/
    ■■■ Interfaces/
    ■■■ Repositories/
    ■■■ Migrations/
    ■■■ DataAccessFactory.cs
```

2.3 Technology Stack

`ASP.NET Web API 2`	HTTP framework for building RESTful APIs
`.NET Framework 4.8`	Application runtime platform
`Entity Framework 6`	ORM — Code First migrations, LINQ queries
`SQL Server (MSSQL)`	Relational database engine
`AutoMapper`	Entity ↔ DTO object mapping
`BCrypt.Net-Next`	Secure password hashing (bcrypt algorithm)
`System.IdentityModel.Tokens.Jwt`	JWT generation and validation
`Newtonsoft.Json`	JSON serialization / deserialization

3. Database Schema

3.1 Entity Relationship Overview

The database contains ten tables. The Employee table is the central entity; nearly all domain tables reference it via foreign keys. The Payroll table aggregates individual PayrollDetail rows — one per employee per month.

Parent Table	Child Table	Cardinality	Notes
Departments	Employees	One → Many	Every employee belongs to one department
Employees	Users	One → One (opt)	Employee may have a login account
Employees	Salaries	One → Many	Salary history over time; only one IsActive
Employees	SalaryRevisions	One → Many	Full audit trail of salary changes
Employees	Deductions	One → Many	Multiple deductions per period
Employees	Bonuses	One → Many	Multiple bonuses per employee
Employees	PayrollDetails	One → Many	Appears in multiple monthly payrolls
Users	SalaryRevisions	One → Many	Tracks who performed the revision
Users	Payrolls	One → Many	Tracks who generated the payroll
Users	RefreshTokens	One → Many	Server-side token store
Payrolls	PayrollDetails	One → Many	One batch, one row per employee

3.2 Table Definitions

Departments

Column	Type	Constraints	Description
DepartmentId	INT	PK, Identity	Auto-incremented primary key
Name	NVARCHAR(100)	NOT NULL	Department name — unique
Description	NVARCHAR(255)	NULL	Optional description
CreatedAt	DATETIME	DEFAULT GETDATE()	Record creation timestamp

Employees

Column	Type	Constraints	Description
EmployeeId	INT	PK, Identity	Auto-incremented primary key
DepartmentId	INT	FK → Departments	Department the employee belongs to
FirstName	NVARCHAR(100)	NOT NULL	Employee first name
LastName	NVARCHAR(100)	NOT NULL	Employee last name
Email	NVARCHAR(150)	NOT NULL, UNIQUE	Contact email — must be unique
Phone	NVARCHAR(20)	NULL	Contact phone number
Address	NVARCHAR(300)	NULL	Residential address
Position	NVARCHAR(100)	NOT NULL	Job title / position
AccountNumber	NVARCHAR(50)	NULL	Bank account for salary disbursement
EmploymentStatus	NVARCHAR(20)	DEFAULT 'Active'	Active | Inactive | Terminated
JoiningDate	DATE	NOT NULL	Date employee joined the organisation
IsActive	BIT	DEFAULT 1	Soft-delete flag
CreatedAt	DATETIME	DEFAULT GETDATE()	Record creation timestamp
UpdatedAt	DATETIME	NULL	Last update timestamp

Users

Column	Type	Constraints	Description
UserId	INT	PK, Identity	Auto-incremented primary key
EmployeeId	INT	FK → Employees (opt)	Linked employee; NULL for admin
Username	NVARCHAR(50)	NOT NULL, UNIQUE	Login username
PasswordHash	NVARCHAR(255)	NOT NULL	BCrypt hashed password
Role	NVARCHAR(20)	NOT NULL	Admin | HR | Employee
IsActive	BIT	DEFAULT 1	Soft-delete / disable flag
CreatedAt	DATETIME	DEFAULT GETDATE()	Account creation timestamp
LastLogin	DATETIME	NULL	Most recent successful login

Salaries

Column	Type	Constraints	Description
SalaryId	INT	PK, Identity	Auto-incremented primary key
EmployeeId	INT	FK → Employees	Owner employee
BasicSalary	DECIMAL(18,2)	NOT NULL	Base monthly salary amount
HouseAllowance	DECIMAL(18,2)	DEFAULT 0	Monthly housing allowance
MedicalAllowance	DECIMAL(18,2)	DEFAULT 0	Monthly medical allowance
TransportAllowance	DECIMAL(18,2)	DEFAULT 0	Monthly transport allowance
EffectiveDate	DATE	NOT NULL	Date this salary record became active
IsActive	BIT	DEFAULT 1	Only one record should be active per employee
Remarks	NVARCHAR(255)	NULL	Optional notes
CreatedAt	DATETIME	DEFAULT GETDATE()	Record creation timestamp

SalaryRevisions

Column	Type	Constraints	Description
RevisionId	INT	PK, Identity	Auto-incremented primary key
EmployeeId	INT	FK → Employees	Employee whose salary was revised
OldSalary	DECIMAL(18,2)	NOT NULL	Basic salary before revision
NewSalary	DECIMAL(18,2)	NOT NULL	Basic salary after revision
RevisionDate	DATE	NOT NULL	Date the revision was effective
Reason	NVARCHAR(255)	NULL	Business reason for revision
RevisedBy	INT	FK → Users	User who performed the revision
CreatedAt	DATETIME	DEFAULT GETDATE()	Audit timestamp

Deductions

Column	Type	Constraints	Description
DeductionId	INT	PK, Identity	Auto-incremented primary key
EmployeeId	INT	FK → Employees	Employee this deduction applies to
DeductionType	NVARCHAR(100)	NOT NULL	Absent | Loan | Penalty | Other
Amount	DECIMAL(18,2)	NOT NULL	Deduction amount
Month	INT	NOT NULL	Calendar month (1–12)
Year	INT	NOT NULL	Calendar year
Reason	NVARCHAR(255)	NULL	Optional explanation
CreatedAt	DATETIME	DEFAULT GETDATE()	Record creation timestamp

Bonuses

Column	Type	Constraints	Description
BonusId	INT	PK, Identity	Auto-incremented primary key
EmployeeId	INT	FK → Employees	Employee this bonus applies to
BonusType	NVARCHAR(100)	NOT NULL	Festival | Performance | Annual | Other
Amount	DECIMAL(18,2)	NOT NULL	Bonus amount
BonusDate	DATE	NOT NULL	Date the bonus was awarded
Reason	NVARCHAR(255)	NULL	Optional explanation
CreatedAt	DATETIME	DEFAULT GETDATE()	Record creation timestamp

Payrolls

Column	Type	Constraints	Description
PayrollId	INT	PK, Identity	Auto-incremented primary key
PayrollMonth	INT	NOT NULL	Calendar month (1–12)
PayrollYear	INT	NOT NULL	Calendar year
GeneratedDate	DATETIME	DEFAULT GETDATE()	When payroll was generated
GeneratedBy	INT	FK → Users	User who generated the payroll
TotalAmount	DECIMAL(18,2)	NOT NULL DEFAULT 0	Sum of all employee net salaries
Status	NVARCHAR(50)	DEFAULT 'Draft'	Draft → Approved → Paid

Note: A UNIQUE constraint exists on (PayrollMonth, PayrollYear) — only one payroll batch per period.

PayrollDetails

Column	Type	Constraints	Description
PayrollDetailId	INT	PK, Identity	Auto-incremented primary key
PayrollId	INT	FK → Payrolls	Parent payroll batch
EmployeeId	INT	FK → Employees	Employee this payslip belongs to
BasicSalary	DECIMAL(18,2)	NOT NULL	Snapshotted basic salary
TotalAllowances	DECIMAL(18,2)	DEFAULT 0	House + Medical + Transport
BonusAmount	DECIMAL(18,2)	DEFAULT 0	Sum of bonuses for this period
TotalDeductions	DECIMAL(18,2)	DEFAULT 0	Sum of deductions for this period
TaxAmount	DECIMAL(18,2)	DEFAULT 0	Calculated tax amount
NetSalary	DECIMAL(18,2)	NOT NULL	GrossSalary - Deductions - Tax
PaymentStatus	NVARCHAR(50)	DEFAULT 'Pending'	Pending | Paid

RefreshTokens

Column	Type	Constraints	Description
TokenId	INT	PK, Identity	Auto-incremented primary key
UserId	INT	FK → Users	Owning user
Token	NVARCHAR(255)	NOT NULL, UNIQUE	Opaque random token string
ExpiryDate	DATETIME	NOT NULL	Expiration timestamp (7 days from issue)
IsRevoked	BIT	DEFAULT 0	Set to 1 on logout or token rotation
CreatedAt	DATETIME	DEFAULT GETDATE()	Issue timestamp

4. Authentication & Security

4.1 JWT Authentication Flow

Every authenticated request must carry a valid JWT access token in the Authorization header. The custom JWTAuthFilter intercepts all protected routes before the controller action executes.

Step	Actor	Action
1	Client	POST /api/auth/login with {username, password}
2	AuthService	Validates credentials via BCrypt.Verify
3	JWTHelper	Generates Access Token (15 min) + Refresh Token (7 days)
4	RefreshTokenRepo	Stores refresh token in RefreshTokens table
5	Client	Receives {accessToken, refreshToken, role, userId, username}
6	Client	Sends API request: Authorization: Bearer <accessToken>
7	JWTAuthFilter	Validates signature, expiry, issuer, audience, and role
8	Controller	Processes request and returns response
9	Client	Access token expires → POST /api/auth/refresh
10	AuthService	Validates refresh token, rotates token pair, revokes old
11	Client	POST /api/auth/logout → refresh token revoked in DB

4.2 Roles & Permissions

Role	Created By	Default Password	Access Level
Admin	SeedAdmin() on startup	Admin@123	Full access to all endpoints
HR	Admin assigns via API	EMP{id}	Employee, salary, payroll management
Employee	Auto-created on employee record creation	EMP{id}	Own data only (future)

4.3 Token Strategy

Token Type	Lifetime	Storage	Purpose
Access Token (JWT)	15 minutes	Client memory / header	Authenticates each API request
Refresh Token (opaque)	7 days	Server DB + client	Issues new access token on expiry

Refresh tokens are rotated on every use — the old token is revoked and a new pair is issued. This limits the window of exposure if a refresh token is compromised. Logging out immediately revokes all active refresh tokens for that user.

5. API Endpoints

All endpoints are prefixed with /api/. Authenticated endpoints require the header: Authorization: Bearer <access_token>

5.1 Auth — api/auth

Method	Endpoint	Description	Access
POST	`/api/auth/login`	Login — returns token pair	Public
POST	`/api/auth/refresh`	Refresh access token	Public
POST	`/api/auth/logout`	Revoke refresh token	Authenticated
PUT	`/api/auth/change-password`	Change own password	Authenticated
PUT	`/api/auth/assign-role`	Assign role to a user	Admin

5.2 Users — api/user

Method	Endpoint	Description	Access
GET	`/api/user/all`	Get all user accounts	Admin
GET	`/api/user/{id}`	Get user by ID	Admin
GET	`/api/user/by-username/{username}`	Get user by username	Admin
PUT	`/api/user/toggle-active/{id}`	Enable / disable user account	Admin
PUT	`/api/user/assign-role`	Assign role to user	Admin

5.3 Departments — api/department

Method	Endpoint	Description	Access
GET	`/api/department/all`	Get all departments	Admin, HR
GET	`/api/department/{id}`	Get department by ID	Admin, HR

Method	Endpoint	Description	Access
POST	`/api/department/create`	Create new department	Admin, HR
PUT	`/api/department/update`	Update department	Admin, HR
DELETE	`/api/department/delete/{id}`	Delete department	Admin, HR

5.4 Employees — api/employee

Method	Endpoint	Description	Access
GET	`/api/employee/all`	Get all active employees	Admin, HR
GET	`/api/employee/{id}`	Get employee by ID	Admin, HR
GET	`/api/employee/search?keyword=...`	Search employees by keyword	Admin, HR
GET	`/api/employee/filter/department/{deptId}`	Filter by department	Admin, HR
GET	`/api/employee/filter/status/{status}`	Filter by employment status	Admin, HR
POST	`/api/employee/create`	Create employee (auto-creates user)	Admin, HR
PUT	`/api/employee/update`	Update employee record	Admin, HR
DELETE	`/api/employee/delete/{id}`	Soft-delete employee	Admin, HR

5.5 Salaries — api/salary

Method	Endpoint	Description	Access
GET	`/api/salary/all`	Get all salary records	Admin, HR
GET	`/api/salary/{id}`	Get salary record by ID	Admin, HR
GET	`/api/salary/employee/{employeeId}`	Get active salary for employee	Admin, HR
GET	`/api/salary/revisions/{employeeId}`	Get salary revision history	Admin, HR
POST	`/api/salary/create`	Create salary record	Admin, HR
PUT	`/api/salary/update`	Update salary (no audit trail)	Admin, HR
POST	`/api/salary/revise`	Revise salary (audit trail)	Admin, HR
DELETE	`/api/salary/delete/{id}`	Soft-delete salary record	Admin, HR

5.6 Bonuses — api/bonus

Method	Endpoint	Description	Access
GET	`/api/bonus/all`	Get all bonuses	Admin, HR
GET	`/api/bonus/{id}`	Get bonus by ID	Admin, HR
GET	`/api/bonus/employee/{employeeId}`	Get bonuses for employee	Admin, HR
POST	`/api/bonus/create`	Create bonus record	Admin, HR
PUT	`/api/bonus/update`	Update bonus record	Admin, HR
DELETE	`/api/bonus/delete/{id}`	Delete bonus record	Admin, HR

5.7 Deductions — api/deduction

Method	Endpoint	Description	Access
GET	`/api/deduction/all`	Get all deductions	Admin, HR
GET	`/api/deduction/{id}`	Get deduction by ID	Admin, HR
GET	`/api/deduction/employee/{employeeId}`	Get deductions for employee	Admin, HR
GET	`/api/deduction/employee/{employeeId}/{month}/{year}`	Get by employee and period	Admin, HR
POST	`/api/deduction/create`	Create deduction record	Admin, HR
PUT	`/api/deduction/update`	Update deduction record	Admin, HR
DELETE	`/api/deduction/delete/{id}`	Delete deduction record	Admin, HR

5.8 Payroll — api/payroll

Method	Endpoint	Description	Access
GET	`/api/payroll/all`	Get all payroll batches	Admin, HR
GET	`/api/payroll/{id}`	Get payroll batch by ID	Admin, HR
GET	`/api/payroll/{id}/details`	Get per-employee payslip breakdown	Admin, HR
POST	`/api/payroll/generate`	Generate payroll for month/year	Admin, HR
PUT	`/api/payroll/approve/{id}`	Approve a Draft payroll	Admin
PUT	`/api/payroll/markpaid/{id}`	Mark an Approved payroll as Paid	Admin
DELETE	`/api/payroll/delete/{id}`	Delete a Draft payroll	Admin, HR

6. Business Logic

6.1 Payroll Calculation Formula

When payroll is generated for a given month and year, the system iterates over all active employees who have an active salary record and computes the following for each:

```
GrossSalary = BasicSalary + HouseAllowance + MedicalAllowance
+ TransportAllowance + BonusAmount

TaxAmount = GrossSalary × 10% (flat rate — configurable)

NetSalary = GrossSalary - TotalDeductions - TaxAmount
```

Bonuses and deductions are summed from the Bonuses and Deductions tables filtered to the target month and year. Employees without an active salary record are skipped. The payroll header TotalAmount is updated to the sum of all employee net salaries after generation.

Payroll Lifecycle

Status	Transition	Who Can Act	Effect
Draft	→ Approved	Admin	Payroll is reviewed and locked for editing
Approved	→ Paid	Admin	All PayrollDetails set to PaymentStatus = Paid
Draft	→ Deleted	Admin, HR	Batch removed (only allowed in Draft state)

6.2 Salary Revision Process

Updating a salary record directly (PUT /api/salary/update) corrects data without creating a history entry. A formal salary revision (POST /api/salary/revise) follows this sequence:

1. Validate that the employee exists and has an active salary record.
2. Verify the new salary is different from the current active salary.
3. Insert a SalaryRevision row recording old value, new value, reason, and who revised it.
4. Set the current active Salary record IsActive = false.
5. Insert a new Salary record with the updated BasicSalary and IsActive = true.

6.3 Auto User Creation on Employee Record

When a new employee record is saved via POST /api/employee/create, the system automatically creates a linked User account:

- Username = firstName.toLower() + employeeId (e.g. john42)
- Password = 'EMP' + employeeId (e.g. EMP42) — BCrypt hashed
- Role = Employee
- IsActive = true

The employee should change their password on first login via PUT /api/auth/change-password.

7. Configuration & Setup

7.1 Prerequisites

Visual Studio	Community 2022 or later with ASP.NET workload
.NET Framework	4.8 Developer Pack
SQL Server	Express or Developer Edition
NuGet	Package restore enabled in Visual Studio

7.2 Database Setup

Update the connection string in API/Web.config:

```
<connectionStrings>
<add name="HRMSDbContext"
connectionString="data source=YOUR_SERVER;
initial catalog=HRMSDb; integrated security=true;"
providerName="System.Data.SqlClient" />
</connectionStrings>
```

Apply migrations from Package Manager Console:

```
Add-Migration InitialCreate -ProjectName DAL
Update-Database -ProjectName DAL
```

7.3 JWT Settings

Configure JWT values in API/Web.config under <appSettings>:

```
<appSettings>
<add key="JwtSecretKey" value="YOUR_SECRET_MIN_32_CHARS" />
<add key="JwtIssuer" value="HRMS" />
<add key="JwtAudience" value="HRMS" />
<add key="JwtAccessTokenMins" value="15" />
<add key="JwtRefreshTokenDays" value="7" />
</appSettings>
```

7.4 CORS Configuration

The API is configured to accept requests from http://localhost:4200 (Angular development server). Update the allowed origin in API/Web.config if your frontend runs on a different port or domain.

8. Example Usage

Login Request

POST /api/auth/login

```
{ "Username": "admin", "Password": "Admin@123" }
```

Response 200 OK

```
{ "AccessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "RefreshToken":
"base64-random-string...", "ExpiresInMinutes": 15, "Role": "Admin", "UserId": 1,
"Username": "admin" }
```

Authenticated Request

```
GET /api/employee/all Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Generate Payroll

POST /api/payroll/generate

```
{ "Month": 5, "Year": 2026, "GeneratedBy": 1 }
```

Revise Salary

POST /api/salary/revise

```
{ "EmployeeId": 3, "NewSalary": 55000.00, "Reason": "Annual increment",
"RevisedBy": 1 }
```

9. Technologies Used

Package / Technology	Version	Purpose
ASP.NET Web API 2	.NET 4.8	HTTP framework — routing, controllers, filters
Entity Framework 6	6.x	ORM — code-first migrations, LINQ, DbContext
AutoMapper	Latest	Entity ↔ DTO mapping with ForMember configuration
BCrypt.Net-Next	4.0.3	Password hashing using bcrypt algorithm
System.IdentityModel.Tokens.Jwt	6.27.0	JWT generation and signature validation
Microsoft.IdentityModel.Tokens	6.27.0	Token validation parameters and security keys
Newtonsoft.Json	Latest	JSON serialisation — StringEnumConverter for enums
System.Configuration.ConfigurationManager	Latest	Reading Web.config appSettings